2.1 Python Libraries to Automate Exploratory Data Analysis

Automate Exploratory Analysis (EDA) is the process of analysing datasets to summarize their main characteristics, often using visual methods. It helps in understanding data before applying modelling techniques.

Exploratory Data Analysis (EDA) is the first step of working with any dataset. The goal is to quickly understand:

- what the dataset contains (rows, columns, types)
- whether the data is clean (missing values, duplicates, wrong types)
- how the values behave (mean, spread, outliers, distribution shape)
- whether there are relationships between variables (correlation, group patterns)

Automating EDA means using libraries that can produce these insights with minimal code, instead of manually checking each column.

2.1.1 Pandas and Numpy

The core of data science in Python relies on Pandas and NumPy for data manipulation, but several other libraries are purpose-built to automate the analysis and reporting part of EDA.

Pandas and NumPy libraries are the foundational pillars of nearly all Python data science projects. They don't automate the entire EDA process, but they provide the essential data structures and functions that automated tools (and manual EDA) rely on.

1. Pandas Library (for table/dataframe operations)

- Pandas is used when your data is in **rows and columns** (like Excel).
- It helps automate EDA by providing one-line functions for summary and cleaning.

Key Features of Pandas Library:

- Pandas Library handles missing values
- Pandas Library does filtering, sorting, merging data

- Pandas Library can provide descriptive statistics (mean, median, mode)
- Easy data loading (CSV, Excel, SQL)

EDA tasks using Pandas Library Function

Pandas Command	Definition	Description
df.head()	Shows the first 5 rows by default (can pass a number like df.head(10))	Column names, sample values, initial data quality (looks correct or not)
df.info()	Displays column data types, non-null counts, and memory usage	Which columns are numeric/text, and where missing values exist
df.describe()	Generates summary statistics for numeric columns (count, mean, std, min, 25%, 50%, 75%, max)	Distribution overview and spread of numeric data
df.isna().sum()	Counts missing values in each column	Missing value count per column
df.duplicated().sum()	Counts duplicate rows	How many repeated records exist
df['col'].value_counts()	Counts frequency of each unique value in a categorical column	Category distribution (most/least common values)
df.groupby('col').mean()	Groups data by a categorical column and calculates mean of numeric columns	Group-wise comparison of averages
df.corr()	Computes correlation matrix for numeric columns	Strength/direction of relationships between numeric variables

Example: EDA using Pandas Library Function

```
import pandas as pd
import numpy as np

# 1) Create a small dataset (like an Excel table)
data = {
   "Name": ["Asha", "Ravi", "Neha", "Amit", "Riya",
"Ravi"],
   "City": ["Surat", "Surat", "Bharuch", "Surat",
"Bharuch", "Surat"],
   "Hours_Studied": [2, 5, 3, 6, np.nan, 5],    # np.nan =
missing value
   "Marks": [35, 78, 55, 88, 60, 78]
}

df = pd.DataFrame(data)

print("=== Dataset ===")
print(df)

# 2) Preview first rows
print("\n=== df.head() ===")
print(df.head())

# 3) Check data types + missing value info
print("\n=== df.info() ===")
df.info()
```

Output:

```
=== Dataset ===
    Name     City  Hours_Studied  Marks
0  Asha    Surat            2.0     35
1  Ravi    Surat            5.0     78
2  Neha  Bharuch            3.0     55
3  Amit    Surat            6.0     88
4  Riya  Bharuch            NaN     60
5  Ravi    Surat            5.0     78

=== df.head() ===
    Name     City  Hours_Studied  Marks
0  Asha    Surat            2.0     35
1  Ravi    Surat            5.0     78
2  Neha  Bharuch            3.0     55
3  Amit    Surat            6.0     88
4  Riya  Bharuch            NaN     60

=== df.info() ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 4 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Name           6 non-null      object
 1   City           6 non-null      object
 2   Hours_Studied  5 non-null      float64
 3   Marks          6 non-null      int64
dtypes: float64(1), int64(1), object(2)
memory usage: 324.0+ bytes

=== df.describe() ===
       Hours_Studied      Marks
count       5.000000   6.000000
mean        4.200000  65.666667
std         1.643168  19.438793
min         2.000000  35.000000
25%         3.000000  56.250000
50%         5.000000  69.000000
75%         5.000000  78.000000
max         6.000000  88.000000
```

```
# 4) Summary stats for numeric columns
print("\n=== df.describe() ===")
print(df.describe())

# 5) Missing values per column
print("\n=== df.isna().sum() ===")
print(df.isna().sum())

# 6) Duplicate rows count
print("\n=== df.duplicated().sum() ===")
print(df.duplicated().sum())

# 7) Frequency count for a categorical column
print("\n=== df['City'].value_counts() ===")
print(df["City"].value_counts())

# 8) Grouped analysis: average marks city-wise
print("\n=== df.groupby('City')['Marks'].mean()
===")
print(df.groupby("City")["Marks"].mean())

# 9) Correlation between numeric columns
print("\n=== df.corr() (numeric columns) ===")
print(df[["Hours_Studied", "Marks"]].corr())
```

```
=== df.isna().sum() ===
Name            0
City            0
Hours_Studied   1
Marks           0
dtype: int64

=== df.duplicated().sum() ===
1

=== df['City'].value_counts() ===
City
Surat      4
Bharuch    2
Name: count, dtype: int64

=== df.groupby('City')['Marks'].mean() ===
City
Bharuch    57.50
Surat      69.75
Name: Marks, dtype: float64

=== df.corr() (numeric columns) ===
               Hours_Studied     Marks
Hours_Studied       1.000000  0.991645
Marks               0.991645  1.000000
```

Overall, **Pandas automates EDA** by helping us understand data quality and patterns (missing values, duplicates, category counts, averages, relationships) in a fast and structured way.

2. NumPy (for fast numeric calculations using arrays)

- NumPy is mainly used for **arrays** (fast numeric containers).
- It is faster than normal Python loops and is heavily used underneath Pandas.

NumPy helps automate EDA by providing:

- fast mean/median/std calculations
- min/max and percentiles
- NaN handling (nanmean, nanstd)
- quick sample data generation (for demos/testing)

EDA functions:

NumPy Function	Meaning	Simple Example	Output Meaning
np.mean()	Finds the **average**	np.mean([10, 20, 30])	Average value (here: 20)
np.median()	Finds the **middle value**	np.median([10, 20, 100])	Middle value (here: 20)
np.std()	Finds the **spread** (variation)	np.std([10, 20, 30])	Higher value = more spread
np.percentile()	Finds a **percentile value**	np.percentile([10,20,30,40], 75)	Value below which 75% data lies

np.isnan()	Checks **which values are NaN**	np.isnan([1, np.nan, 3])	True means missing (NaN)
np.nanmean()	Mean **ignoring NaN**	np.nanmean([10, np.nan, 30])	Average without NaN (here: 20)

Example: Performing EDA tasks using Numpy Library

```
import numpy as np

# 1) NumPy array (numeric data)
arr = np.array([10, 12, 15, 18, 20, 22, 25])
print("=== Array ===")
print(arr)

# Mean, Median, Standard Deviation
print("\n=== Mean / Median / Std Dev ===")
print("Mean   :", np.mean(arr))
print("Median :", np.median(arr))
print("Std Dev:", np.std(arr))

# Min, Max
print("\n=== Min / Max ===")
print("Min:", np.min(arr))
print("Max:", np.max(arr))

# Percentiles
print("\n=== Percentiles ===")
print("25th percentile:", np.percentile(arr, 25))
print("50th percentile:", np.percentile(arr, 50))
print("75th percentile:", np.percentile(arr, 75))

# 2) Handling NaN values
arr_nan = np.array([10, 12, np.nan, 18, 20])
print("\n=== Array with NaN ===")
print(arr_nan)

print("\n=== np.isnan() (detect missing values) ===")
print(np.isnan(arr_nan))

print("\n=== Mean vs NaN-mean ===")
print("np.mean(arr_nan)    :", np.mean(arr_nan))     #
becomes nan
print("np.nanmean(arr_nan) :", np.nanmean(arr_nan))   #
ignores nan


print("\n=== Sample stats (mean, min, max) ===")
print("Mean:", np.mean(sample))
print("Min :", np.min(sample))
print("Max :", np.max(sample))
```

Output:

```
=== Array ===
[10 12 15 18 20 22 25]

=== Mean / Median / Std Dev ===
Mean   : 17.428571428571427
Median : 18.0
Std Dev: 5.010193690500052

=== Min / Max ===
Min: 10
Max: 25

=== Percentiles ===
25th percentile: 13.5
50th percentile: 18.0
75th percentile: 21.0

=== Array with NaN ===
[10. 12. nan 18. 20.]

=== np.isnan() (detect missing values) ===
[False False  True False False]

=== Mean vs NaN-mean ===
np.mean(arr_nan)    : nan
np.nanmean(arr_nan) : 15.0

=== Generate sample data (random integers) ===
[72 28 64 65 59 21  6 76 17 77]

=== Sample stats (mean, min, max) ===
Mean: 48.5
Min : 6
Max : 77
```

Example: Performing EDA tasks using both Pandas & Numpy Library Functions

```
import pandas as pd
import numpy as np

# NumPy arrays (fast numeric data)
hours = np.array([2, 5, 3, 6, np.nan, 5], dtype=float)   # NaN = missing value
marks = np.array([35, 78, 55, 88, 60, 78], dtype=int)

# Pandas DataFrame (table) built using NumPy arrays
df = pd.DataFrame({
   "Name": ["Asha", "Ravi", "Neha", "Amit", "Riya", "Ravi"],
   "City": ["Surat", "Surat", "Bharuch", "Surat", "Bharuch", "Surat"],
   "Hours_Studied": hours,
   "Marks": marks
})

print("=== DataFrame (Pandas) built from NumPy arrays ===")
print(df)

# 1) Quick EDA with Pandas
print("\n=== Missing values per column (Pandas) ===")
print(df.isna().sum())

print("\n=== Summary stats (Pandas) ===")
print(df.describe().to_string())

# 2) Handle NaN using NumPy, then apply back in Pandas
mean_hours = np.nanmean(df["Hours_Studied"].to_numpy())  # NumPy ignores NaN
df["Hours_Studied"] = df["Hours_Studied"].fillna(mean_hours)

print("\n=== Filled missing Hours_Studied using np.nanmean() ===")
print("Mean Hours (ignoring NaN):", mean_hours)
print(df)

# 3) Create a new column using NumPy logic (vectorized)
df["Result"] = np.where(df["Marks"] >= 50, "Pass", "Fail")

print("\n=== Added Result column using np.where() ===")
print(df[["Name", "Marks", "Result"]])

# 4) Group analysis using Pandas
print("\n=== City-wise average marks (Pandas groupby) ===")
print(df.groupby("City")["Marks"].mean())
```

Output:

```
=== DataFrame (Pandas) built from NumPy arrays ===
   Name     City  Hours_Studied  Marks
0  Asha    Surat            2.0     35
1  Ravi    Surat            5.0     78
2  Neha  Bharuch            3.0     55
3  Amit    Surat            6.0     88
4  Riya  Bharuch            NaN     60
5  Ravi    Surat            5.0     78

=== Missing values per column (Pandas) ===
Name            0
City            0
Hours_Studied   1
Marks           0
dtype: int64

=== Summary stats (Pandas) ===
       Hours_Studied      Marks
count       5.000000   6.000000
mean        4.200000  65.666667
std         1.643168  19.438793
min         2.000000  35.000000
25%         3.000000  56.250000
50%         5.000000  69.000000
75%         5.000000  78.000000
max         6.000000  88.000000

=== Filled missing Hours_Studied using np.nanmean() ===
Mean Hours (ignoring NaN): 4.2
   Name     City  Hours_Studied  Marks
0  Asha    Surat            2.0     35
1  Ravi    Surat            5.0     78
2  Neha  Bharuch            3.0     55
3  Amit    Surat            6.0     88
4  Riya  Bharuch            4.2     60
5  Ravi    Surat            5.0     78


=== Added Result column using np.where() ===
   Name  Marks Result
0  Asha     35   Fail
1  Ravi     78   Pass
2  Neha     55   Pass
3  Amit     88   Pass
4  Riya     60   Pass
5  Ravi     78   Pass

=== City-wise average marks (Pandas groupby) ===
City
Bharuch    57.50
Surat      69.75
Name: Marks, dtype: float64
```

Difference Between Pandas and NumPy Libary

Difference Point	Pandas	NumPy
Best suited for	Best for tabular data (rows and columns like Excel/CSV)	Best for numerical data (numbers and mathematical operations)
Main data tools	Uses DataFrame (table) and Series (single column)	Uses Arrays (ndarray)
Memory usage	Generally uses more memory	Generally more memory-efficient
Performance in practice	Often preferred for very large tables and data operations like filtering/grouping	Often faster for pure numeric computation and array math

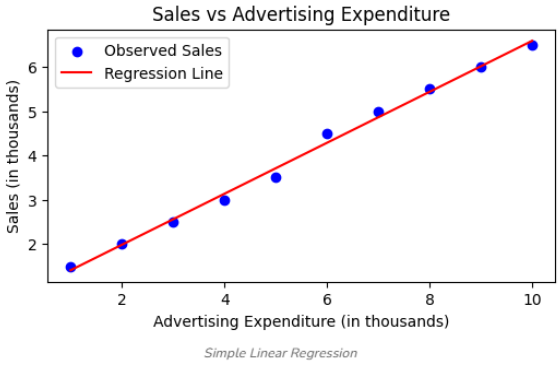
Indexing speed	Indexing in Series/DataFrame can be slower than raw arrays	Indexing in arrays is usually very fast
Data shape support	Mainly designed for 2D tables (DataFrame), also supports 1D (Series)	Supports multi-dimensional arrays (1D, 2D, 3D, …)
Background	Developed by **Wes McKinney**; initial public release widely cited around **2008**	Developed by **Travis Oliphant**; initial public release widely cited around **2005–2006**

2.2 Regression

Regression is a statistical and machine learning technique used to **predict a numeric (continuous) value** based on one or more input variables.

- If the output is a **number** (price, marks, salary, temperature, sales), it is usually a **regression** problem.
- If the result to be predicted is a numeric value such as price, marks, salary, temperature, or sales, then it is treated as a regression problem. Regression also helps to understand which input affects the output more and how much it affects it.

In simple words, regression finds a relationship where the output (Y) depends on the input (X). For example, house price depends on area, marks depend on study hours, and sales depend on advertising budget.



Simple Linear Regression

Regression finds a relationship like: Output (Y) depends on Input (X)

Example:

- House Price depends on Area
- Marks depend on Study Hours
- Sales depend on Advertising Budget

Types of Regression

1) Simple Linear Regression
This type uses **one input** to predict **one output**.

Example: Marks depend on study hours.
So, **Marks = (Hours)**.

2) Multiple Linear Regression
This type uses **many inputs** to predict **one output**.
Example: House price depends on area, location, and number of bedrooms.
So, **House Price = f(Area, Location, Bedrooms)**.

3) Non-Linear Regression
This type is used when the relationship is **not a straight line** and becomes **curved**.
Example: Growth or decay patterns, where values increase or decrease in a curve (like population growth or medicine effect saturation).

2.2.1 Characteristics of Regression

1. **Regression is used to predict a number (continuous output).**
   Regression is applied when the output is a numeric value that can take many possible values, such as marks, salary, house price, temperature, profit, or sales. The output is not a category like pass/fail or yes/no. If the output is a category, then it becomes a classification problem, not regression.
2. **Regression learns a relationship between input and output.**
   Regression studies the pattern between input variables and the output variable. It answers questions like: "If the input increases, does the output increase or decrease?" and "How much change in output happens for a small change in input?" This relationship can be strong, weak, positive, or negative depending on the dataset.
3. **Regression uses independent variables and a dependent variable.**
   The inputs are called **independent variables (X)** because they are used as predictors. The output is called the **dependent variable (Y)** because its value depends on the input values.
   Example: Hours Studied (X) is used to predict Marks (Y).
   In multiple regression, there can be many X variables, but usually only one Y variable.
4. **Regression fits a best-fit line or best-fit curve to the data.**
   Regression does not try to make all points match perfectly. Instead, it creates a mathematical line or curve that represents the overall trend. The line/curve is chosen so that the total prediction error across all points becomes as small as possible. Linear regression creates a straight line, while non-linear regression creates a curved shape.
5. **Regression produces prediction errors (residuals).**
   For each data record, the model gives a predicted value. The difference between the actual value and predicted value is called the **error** or **residual**.

   $\text{Error} = Y_{actual} - Y_{predicted}$

   A good regression model keeps these errors small for most data points.
6. **Regression works with one input or many inputs.**
   If only one input variable is used, it is called **simple regression**. If multiple input variables are used, it is called **multiple regression**.
   Example:
   - Simple: Marks depend on Hours Studied
   - Multiple: House price depends on Area, Location, Bedrooms
7. **Regression can be linear or non-linear depending on the pattern.**

- **Linear regression** is used when data follows a straight-line trend.
- **Non-linear regression** is used when data shows a curved trend, such as growth, decay, or saturation.
  Example of saturation: after a point, increasing input does not increase output much.

8. **Regression depends on clean and correct data.**
   Regression gives better results when data is cleaned properly. Missing values, wrong data types, duplicate rows, and incorrect entries can reduce prediction quality. Before applying regression, typical cleaning steps include handling missing values, removing duplicates, correcting data types, and checking unrealistic values.
9. **Regression is sensitive to outliers.**
   Outliers are extreme values that are far from the normal pattern. For example, if most house prices are between 40–80 lakhs but one house price is 5 crores, that entry may act as an outlier. Outliers can pull the best-fit line/curve away from the correct trend and reduce accuracy for normal values.
10. **Regression is evaluated using error-based measures (not accuracy).**
    Regression does not use "accuracy" like classification. Instead, it uses metrics such as:
    - **MAE (Mean Absolute Error):** average size of mistakes
    - **MSE (Mean Squared Error):** squares mistakes and gives higher penalty to big errors
    - **RMSE:** square root of MSE, easy to understand because it is in the same unit as output
    - **R^2 (R-squared):** shows how well input variables explain the output
      Lower MAE/RMSE usually means better model performance.
11. **Regression requires training and testing.**
    A regression model is trained using training data and then tested using new or unseen data. This is done to confirm the model works in real situations. If performance is good only on training data but poor on testing data, the model is not reliable for prediction.
12. **Regression can face underfitting and overfitting problems.**
    - **Underfitting** happens when the model is too simple and cannot learn the true pattern.
    - **Overfitting** happens when the model learns training data too deeply (including noise) and then fails on new data.
      A good regression model should learn the real pattern and perform well on unseen data.
13. **Regression can be used for both prediction and understanding.**
    Regression is useful not only for predicting values but also for understanding which input factor affects the output more. In linear regression, coefficients show the impact of each input variable. This helps in decision-making, such as understanding whether advertising budget strongly affects sales.

For Example: Dataset of study hours and marks:

- Hours Studied (X): 1, 2, 3, 4, 5
- Marks (Y): 40, 50, 60,70, 80

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
 	
# Simple data: Hours studied (X) and Marks (Y) 	
x = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([40, 50, 60, 70, 80], dtype=float)
	Output:
	
	Regression equation: Marks = 30.0 + 10.0 * Hours
	
	Prediction for 6 hours: 90.0

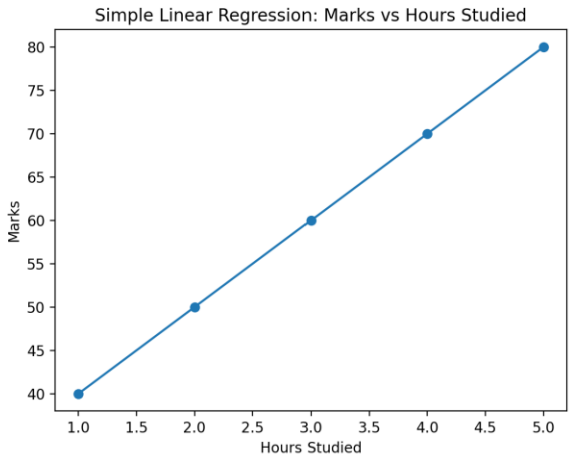
```
# Regression line: y = a + b*x
b = np.sum((x - x.mean()) * (y - y.mean())) /
np.sum((x - x.mean())**2)  # slope
a = y.mean() - b * x.mean()
# intercept

print("Regression equation: Marks =", a, "+", b,
"* Hours")

# Predict marks for 6 hours
hours = 6
pred = a + b * hours
print("Prediction for 6 hours:", pred)

#ploting

y_pred = a + b * x
plt.figure()
plt.scatter(x, y)
plt.plot(x, y_pred)
plt.title("Simple Linear Regression: Marks vs
Hours Studied")
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.show()
```



This pattern shows that when study hours increase, marks also increase. Regression learns this trend and forms a best-fit line. After training, if a new student studies **6 hours**, regression uses the learned relationship to predict the marks. The predicted value might be around **90 (approx)**.

This is regression because the output (marks) is a numeric value and the goal is to predict a continuous number based on input values.

2.2.2 Dependent and Independent Variables

In regression, two types of variables are used to build a prediction model. One type is the **input** (cause/predictor), and the other type is the **output** (result/target).

1) Independent Variables (X)

Independent variables are the **input variables** used to predict the output. They are also called **predictors**, **features**, or **explanatory variables**. These variables are considered "independent" because their values are given, and they are used to explain the change in the output.

Examples of independent variables

- Study hours, attendance, class test score
- House area, number of bedrooms, location rating
- Advertising budget, discount percentage, season
- Age, experience, qualification

Independent variables can be:

- **One** (in simple regression)
- **Many** (in multiple regression)

2) Dependent Variable (Y)

The dependent variable is the **output variable** that needs to be predicted. It is called "dependent" because its value depends on the independent variables.

Examples of dependent variables

- Marks (depends on hours studied, attendance)
- House price (depends on area, location)
- Sales (depends on advertising budget, discount)
- Salary (depends on experience, skills)

3) Identification of X and Y in a question

A simple rule is:

- The variable that is being **predicted** is the **dependent variable (Y)**.
- The variables that are used to **predict** are **independent variables (X)**.

Mathematically:

$Y = f(X)$

Where Y = Dependent variable, X = Independent variable

Example 1: Predict marks using study hours

- X = Hours Studied
- Y = Marks

Example 2: Predict house price using area, bedrooms, location

- X = Area, Bedrooms, Location
- Y = House Price

Independent variables are used as the "reasons" or "factors," and the dependent variable is the "result." Regression learns how each independent variable affects the dependent variable. Understanding these two types of variables is fundamental to regression analysis.

Example: Independent vs Dependent Variable (with Regression)

Scenario: Electricity Bill depends on Units Consumed

- **Independent Variable (X):** Units Consumed (input)
- **Dependent Variable (Y):** Bill Amount (output)

```
import numpy as np
import matplotlib.pyplot as plt

# Independent variable (X): Units Consumed
x = np.array([50, 100, 150, 200, 250],
dtype=float)

# Dependent variable (Y): Bill Amount (₹)
y = np.array([300, 550, 800, 1050, 1300],
dtype=float)

# Linear Regression (no sklearn): y = a + b*x
b = np.sum((x - x.mean()) * (y - y.mean())) /
np.sum((x - x.mean())**2)  # slope
a = y.mean() - b * x.mean()
# intercept

print("Independent (X): Units Consumed")
print("Dependent   (Y): Bill Amount")
print(f"Regression equation: Bill = {a:.2f} +
{b:.2f} * Units")

# Predict bill for 180 units
units = 180
pred = a + b * units
print(f"Predicted bill for {units} units:
₹{pred:.2f}")

# Plot: scatter + regression line
y_pred = a + b * x
plt.figure()
plt.scatter(x, y)
plt.plot(x, y_pred)
plt.title("Regression: Bill Amount vs Units
Consumed")
plt.xlabel("Units Consumed (X)")
plt.ylabel("Bill Amount (Y)")
plt.show() Amount (output)
```

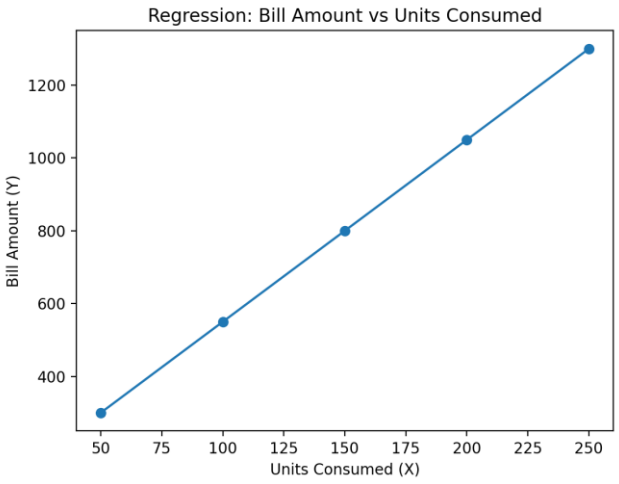
Output:

Independent (X): Units Consumed

Dependent (Y): Bill Amount

Regression equation: Bill = 50.00 + 5.00 * Units

Predicted bill for 180 units: ₹950.00



Variable Type	Definition	Role in Regression	Example
Dependent Variable (Y)	The variable we are trying to **predict** or explain.	The **target** or **outcome** variable.	**House Price**
Independent Variable (X)	The variable(s) used to **predict** the dependent variable.	The **predictor** or **feature** variable(s).	**Square Footage, Number of Bedrooms**

2.2.3 Covariance and Correlation

Covariance and correlation are used in EDA to understand the relationship between two variables, especially numeric variables.

A) Covariance

Covariance tells whether two variables change together.

- **Positive covariance:** both variables increase together (same direction)
  Example: Study hours ↑ and marks ↑
- **Negative covariance:** one increases and the other decreases (opposite direction)
  Example: Product price ↑ and demand ↓
- **Near Zero Covariance :** no clear linear relationship

The unit of covariance is the product of the units of the two variables.

$$Cov(X,Y) = \frac{\sum_{i=1}^{n}(X_i - \bar{X})(Y_i - \bar{Y})}{n-1}$$

X bar is simple mean (average) of the X variable. Same for Y bar.

Covariance shows the **direction** of relationship, but it does not clearly show the **strength** because its value depends on the units of data. For example, covariance changes if marks are measured out of 100 or out of 50.

B) Correlation

Correlation is a standardized measure of the **strength and direction** of the relationship between two variables. Its value is always between **-1 and +1**.

- **+1:** perfect positive relationship
- **0:** no linear relationship
- **-1:** perfect negative relationship

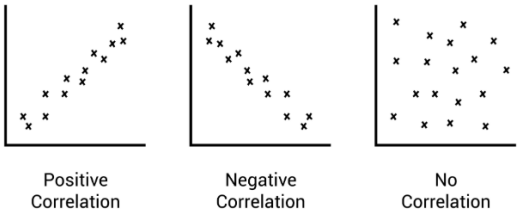
Interpretation

- Correlation near **+1** means both variables move together strongly.
- Correlation near **-1** means they move oppositely strongly.
- Correlation near **0** means there is no strong straight-line relationship.
- For more details as follows:

$$\text{Correlation } (r) = \frac{\text{Covariance}(X,Y)}{\text{Standard Deviation}(X) \times \text{Standard Deviation}(Y)}$$

Correlation Value (r)	Interpretation
$r = +1$	Perfect positive linear relationship.
$0 < r < +1$	Positive linear relationship (as one increases, the other increases).
$r = 0$	No linear relationship.
$-1 < r < 0$	Negative linear relationship (as one increases, the other decreases).
$r = -1$	Perfect negative linear relationship.

A **visual representation of correlation types** using scatter plots



Example: Suppose there are students with study hours and marks:

Hours (X)	Marks (Y)
2	40
3	55
4	65
5	80

In this data, when hours increase, marks also increase. So:

- Covariance will be **positive** (same direction).
- Correlation will be **positive** and likely **high**, showing a strong positive relationship.

Example:

```
import numpy as np
import pandas as pd

# ------------------------------
# 1) dataset
# ------------------------------
df = pd.DataFrame({
   "Hours": [1, 2, 3, 4, 5, 6, 7, 8],              # Independent variable (X)
   "Marks": [32, 40, 45, 55, 60, 68, 75, 82]         # Dependent variable (Y)
})

print("=== Dataset ===")
print(df)

# ------------------------------
# 2) Dependent (Y) and Independent (X)
# ------------------------------
X = df["Hours"].to_numpy()
Y = df["Marks"].to_numpy()

print("\n=== Independent and Dependent Variables ===")
print("Independent (X) = Hours :", X)
print("Dependent   (Y) = Marks :", Y)

# ------------------------------
```

Output:

```
=== Dataset ===
   Hours  Marks
0      1     32
1      2     40
2      3     45
3      4     55
4      5     60
5      6     68
6      7     75
7      8     82

=== Independent and Dependent Variables ===
Independent (X) = Hours : [1 2 3 4 5 6 7 8]
Dependent   (Y) = Marks : [32 40 45 55 60 68 75 82]

=== Covariance and Correlation (NumPy) ===
Covariance matrix:
 [[  6.          42.78571429]
 [ 42.78571429 305.83928571]]
Covariance (Hours, Marks): 42.785714285714285
Correlation matrix:
 [[1.         0.99879538]
 [0.99879538 1.        ]]
Correlation (Hours, Marks): 0.9987953830717263

=== Covariance and Correlation (Pandas) ===
df.cov():
            Hours       Marks
Hours   6.000000   42.785714
Marks  42.785714  305.839286
df.corr():
            Hours     Marks
Hours   1.000000  0.998795
Marks   0.998795  1.000000
```

```
# 3) Covariance and Correlation
# ----------------------------
# Sample covariance matrix (ddof=1)
cov_matrix = np.cov(X, Y)
cov_xy = cov_matrix[0, 1]

# Correlation coefficient matrix
corr_matrix = np.corrcoef(X, Y)
corr_xy = corr_matrix[0, 1]

print("\n=== Covariance and Correlation (NumPy) ===")
print("Covariance matrix:\n", cov_matrix)
print("Covariance (Hours, Marks):", cov_xy)
print("Correlation matrix:\n", corr_matrix)
print("Correlation (Hours, Marks):", corr_xy)

print("\n=== Covariance and Correlation (Pandas) ===")
print("df.cov():\n", df.cov())
print("df.corr():\n", df.corr(numeric_only=True))
```

Difference Between Covariance vs Correlation

Aspect	Covariance	Correlation
Meaning	Measures how two variables vary together	Measures how strongly two variables move together (linear relationship)
Direction	Shows direction (positive/negative)	Shows direction (positive/negative)
Strength	Does **not** clearly show strength (magnitude depends on scale)	Shows **strength** clearly
Range	No fixed range (can be any value)	Fixed range: **−1 to +1**
Standardization	Not standardized	Standardized
Units	Has units (Unit of X × Unit of Y)	Unit-less
Comparability	Hard to compare across different variables/datasets	Easy to compare across different variables/datasets
Scale effect	Changes if you change units (cm → m)	Does not change with unit conversion
Best use	Understanding joint variation in original units	Comparing relationships and interpreting strength easily
Relationship	—	**Correlation = Covariance / (σx · σy)**

2.3 Machine Learning Basics

Machine Learning (ML) is a part of Artificial Intelligence (AI) where a computer system learns from data and improves its performance without being explicitly programmed for every rule. Instead of writing "if-else" rules for all situations, a machine learning model learns patterns from past examples and uses those patterns to make predictions or decisions on new data.

2.3.1 Concepts of Machine Learning

Machine learning mainly works on three ideas. The first idea is that data is used as input. The second idea is that an algorithm learns patterns from that data. The third idea is that the learned model is used to predict or classify new data. Machine learning problems are generally of three types. In **supervised learning**, the dataset contains input and the correct output (label), and the model learns to predict that output. In **unsupervised**

learning, only input data is given and the model finds hidden patterns like grouping. In **reinforcement learning**, the system learns by trial and error using rewards and penalties.

In simple terms, instead of a programmer writing thousands of lines of code with explicit rules (e.g., "If the customer's age is less than 25 AND they live in a city, THEN offer them a youth discount"), a machine learning model is fed a large amount of historical data (e.g., customer age, location, and whether they took a discount). The algorithm then automatically figures out the complex patterns and rules that best predict the outcome.

In simple terms Machine Learning = Learning from Data + Making Predictions/Decisions

Example:

- Netflix recommending movies
- Gmail filtering spam
- A bank predicting loan defaults
- Google Maps suggesting fastest routes

The formal definition of a computer program learning from experience, provided by **Tom M. Mitchell**, is:

"A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."

- **Example:** Teaching a computer to identify cats in pictures.
    - **Task (T):** Classifying an image as "Cat" or "Not Cat."
    - **Experience (E):** Showing the computer thousands of labeled pictures of cats and other animals.
    - **Performance (P):** The percentage of new, unseen pictures the computer correctly labels (its accuracy).

2.3.1.1 Understanding Machine Learning

Machine learning can be understood as “learning from experience.” The experience is the data. When a model is trained on a dataset, it learns relationships between input features and output results. After training, the model can take new input data and give an output prediction.

For example, if a dataset contains “study hours” and “marks,” a machine learning model can learn how marks depend on study hours. After learning, it can predict marks for a new student based on study hours. Similarly, a model can learn from past house prices and predict the price of a new house based on area, location, and other features.

A machine learning model is not always perfect. It makes predictions based on patterns in the data. If the data is poor or biased, the model’s predictions will also be poor. That is why data quality is very important in machine learning.

ML vs. Traditional Programming

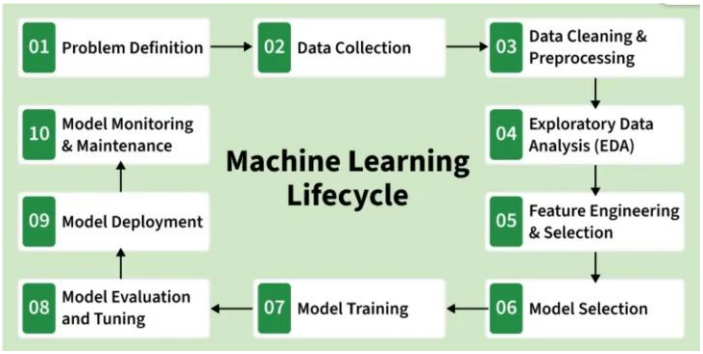
Feature	Traditional Programming	Machine Learning
Input	Data and explicit Program Rules	Data and desired Output (or rewards)
Output	The Answer/Output	The **Program** (the trained model)
Learning	No learning; deterministic execution of code.	Learns patterns from data to generate a model.

2.3.1.2 Benefit of Machine Learning

1. Machine learning can handle **very large datasets** and process thousands or millions of records efficiently.
2. Machine learning helps in **better decision-making** by giving predictions based on past data instead of guesswork.
3. Machine learning can **automate repetitive tasks**, such as sorting emails, checking documents, or detecting errors.
4. Machine learning models can **learn and improve over time** when more data is collected and used for training.
5. Machine learning reduces **human effort and time** by doing analysis and prediction automatically.
6. Machine learning can work with **different types of data**, such as numbers, text, images, audio, and videos.
7. Machine learning can provide **faster results** than manual analysis, especially when the dataset is large.
8. Machine learning can give **consistent results**, meaning it follows the same logic every time and reduces human mistakes.
9. Machine learning supports **personalization**, such as recommending content based on user interest and history.
10. Machine learning is useful for **forecasting**, such as predicting sales, demand, weather trends, or stock movement patterns.
11. Machine learning is widely used in **spam email detection** to identify unwanted or harmful emails automatically.
12. Machine learning is used in **product recommendation systems**, such as recommending products on shopping sites or movies on OTT platforms.
13. Machine learning is used in **fraud detection** in banking and online payments by identifying unusual transactions.
14. Machine learning is used in **face recognition** systems for attendance, phone unlocking, and security verification.
15. Machine learning is used in **voice assistants** and speech recognition to understand commands and respond.

2.3.1.3 Machine Learning Life Cycle

The machine learning life cycle is the step-by-step process followed to build a machine learning solution.



Step 1: Problem Definition

The first step is to identify and clearly define the business problem. A properly framed problem becomes the base of the complete machine learning lifecycle. Project objectives, expected outcomes, and the scope of the work are decided at this stage.

- Stakeholders are consulted to understand business goals.
- Project objectives, scope, and success criteria are defined.
- The desired output and expected result are made clear.

Step 2: Data Collection

This step involves collecting datasets that will be used as raw data for training the model. The performance of a machine learning model strongly depends on the quality and variety of collected data.

- **Relevance:** Collected data should match the defined problem and include required features.
- **Quality:** Data should be accurate and collected/used ethically.
- **Quantity:** Sufficient amount of data should be collected for reliable training.
- **Diversity:** Data should include different cases and scenarios to represent real-world patterns.

Step 3: Data Cleaning and Preprocessing

Raw data is usually incomplete, noisy, and unstructured. Training a model directly on such data can reduce accuracy. Therefore, data cleaning and preprocessing are performed.

- **Data Cleaning:** Missing values, outliers, and inconsistent data are corrected or handled.
- **Data Preprocessing:** Formats are standardized, numeric values are scaled, and categorical values are encoded.
- **Data Quality:** Data is organized properly so that meaningful analysis and learning can happen.

Step 4: Exploratory Data Analysis (EDA)

EDA is used to understand the dataset and discover hidden patterns. It helps in finding trends, relationships, and problems in the data that are not easily visible. The insights from EDA support better decisions in later steps.

- **Exploration:** Statistical methods and charts are used to understand the data.
- **Patterns and Trends:** Relationships, trends, and possible challenges are identified.
- **Insights:** Useful observations are generated for better decisions.
- **Decision Making:** EDA supports feature engineering and model selection.

Step 5: Feature Engineering and Selection

This step focuses on improving input data by creating better features and selecting only the important ones. It helps the model learn better and reduces unnecessary complexity.

- **Feature Engineering:** New features are created or existing features are transformed to represent patterns better.
- **Feature Selection:** Only the most useful features are chosen to improve performance.
- **Domain Expertise:** Subject knowledge is used to design meaningful features.
- **Optimization:** A balance is maintained between good accuracy and low complexity.

Step 6: Model Selection

In this step, an appropriate algorithm is chosen based on the problem type, data characteristics, and expected outcomes. Different models may perform differently, so selection is important.

- **Complexity:** Model choice depends on data size and problem difficulty.
- **Decision Factors:** Performance, interpretability, and scalability are considered.
- **Experimentation:** Multiple models are tested to find the best one.
-

Step 7: Model Training

Model training is the process where the chosen model learns from historical data. During training, the model learns patterns and relationships between input and output.

- **Iterative Process:** Training happens repeatedly to reduce errors.
- **Optimization:** Parameters are adjusted to improve prediction accuracy.
- **Validation:** Training is done carefully to ensure performance on unseen data.

Step 8: Model Evaluation and Tuning

The trained model is tested on validation or test data to check how well it performs on new data. If the performance is not satisfactory, tuning is done by changing hyperparameters and improving the model.

- **Evaluation Metrics:** Measures such as accuracy, precision, recall, and F1-score are used (based on the problem type).
- **Strengths and Weaknesses:** Testing shows where the model performs well and where it fails.
- **Iterative Improvement:** Hyperparameters are tuned to improve performance.
- **Model Robustness:** Tuning helps achieve stable and reliable predictions.

Step 9: Model Deployment

After final evaluation, the model is deployed for real-world use. Deployment means connecting the model with applications so predictions can be used for decision-making.

- The model is integrated with existing systems.
- Predictions are used for practical decision-making.
- Deployment is made scalable and secure.
- APIs or pipelines are provided for production use.

Step 10: Model Monitoring and Maintenance

After deployment, the model must be continuously monitored to ensure it stays accurate over time. Real-world data can change, so performance may drop and retraining may be required.

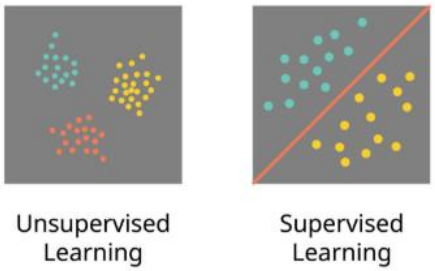
- Model performance is tracked regularly.
- Data drift or concept drift is detected.
- Retraining is done when accuracy decreases.
- Logs and alerts are maintained to detect real-time issues.

Real-Life Example: Predicting Electricity Bill from Units (Home/Hostel)

Step	What it means	Real-life example (Electricity bill prediction)
01. Problem Definition	Decide the goal	Predict next month's **electricity bill** so the family can plan budget and reduce usage.
02. Data Collection	Collect data	Gather last 12–24 months data: **units consumed**, bill amount, month, number of people at home, AC usage.
03. Data Cleaning & Preprocessing	Make data usable	Remove missing months, correct wrong entries (e.g., units = 0), convert month names to numbers, scale units if needed.
04. Exploratory Data Analysis (EDA)	Understand patterns	Check: bills higher in summer? relationship between units and bill? detect outliers (party/guests month).
05. Feature Engineering & Selection	Create helpful features	Add features like: **Summer/Winter flag**, AC-on days, weekend usage, tariff slab category.
06. Model Selection	Choose model	Start with Linear Regression; if non-linear, try Decision Tree / Random Forest.
07. Model Training	Train the model	Train model using past months data so it learns the pattern from units to bill.
08. Model Evaluation & Tuning	Test and improve	Use MAE/RMSE to see average error; tune features or model settings to improve accuracy.
09. Model Deployment	Put into use	Create a simple app/Excel/Python script: enter units → get predicted bill instantly.
10. Model Monitoring & Maintenance	Keep updating	If tariff changes or new appliances added (AC, geyser), accuracy drops → retrain with latest data.

2.4 Types of Machine Learning

Machine learning can be divided into types based on **how the model learns from data**. Some methods learn using data that already has the correct answers, while other methods learn without any answers and try to discover patterns. The two most common and important types are **Supervised Learning** and **Unsupervised Learning**. These are widely used in real-world applications.



2.4.1 Supervised and Unsupervised Learning

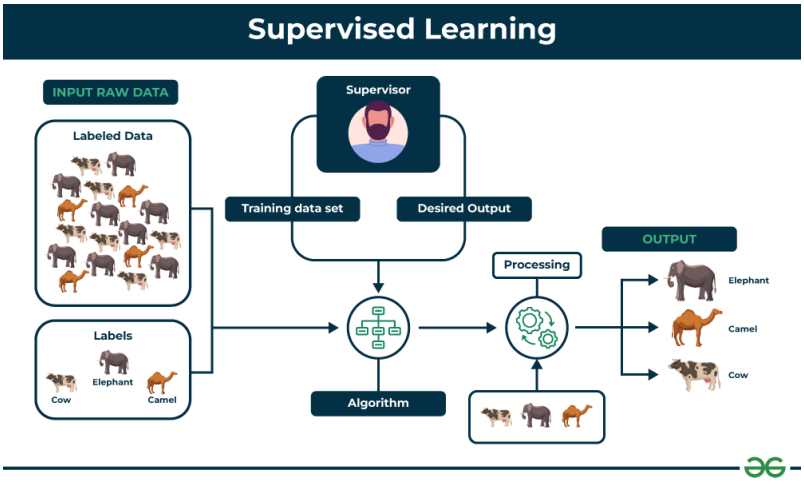
Machine Learning models learn patterns from data to solve problems such as prediction, classification, grouping, and discovery of hidden trends. Based on the type of data available and the goal of learning, machine learning is commonly divided into Supervised Learning and Unsupervised Learning. The main difference between these two approaches is the availability of the correct output (label) in the dataset.

A) Supervised Learning

Supervised learning is a type of machine learning in which the training dataset contains both input data (X) and the correct output (Y). The output is called a label or target. Since the model learns using correct answers provided in the dataset, it is called “supervised.”

In supervised learning, the model is trained using many examples. For each example, the model receives the input features and the correct output label. During training, the model makes predictions and compares them with the correct output. The difference between the prediction and the true output is treated as an error, and the model learns to reduce this error. After training, the model can predict the correct output for new input data that it has never seen before.

For example, a labeled dataset of images of Elephant, Camel and Cow would have each image tagged with either "**Elephant**", "**Camel**" or "**Cow**."



Key characteristics of Supervised Learning

1. Labeled data is required. Each record contains input features and a correct output label.

2. The main goal is prediction. The model learns a mapping from inputs to outputs (X → Y).
3. Training and testing are necessary. The model is trained on training data and evaluated on testing data to check how well it performs on unseen data.
4. It is used when past output is known. Many real-world datasets contain historical inputs and their correct outputs, which makes supervised learning practical.

Types of supervised learning problems

Supervised learning problems are mainly of two types:

1) Classification

Classification is used when the output is a category or class. The model predicts a label rather than a number. Examples of classification outputs include:

- Spam / Not Spam
- Pass / Fail
- Positive / Negative sentiment
- Disease / No Disease

Classification is used whenever the final answer is a class name or category.

Example: Predict whether a student will **Pass** or **Fail** based on **Study Hours**. This is **supervised learning** because the dataset has **input (Hours)** and **label/output (Pass/Fail)**.

```
from sklearn.tree import DecisionTreeClassifier
import pandas as pd

# Training data (Hours -> Result)
data = {
   "Hours":  [1, 2, 3, 4, 5],
   "Result": ["Fail", "Fail", "Pass", "Pass", "Pass"]
}
df = pd.DataFrame(data)

X = df[["Hours"]]     # input
y = df["Result"]      # output label

# Train model
model = DecisionTreeClassifier()
model.fit(X, y)

# Predict for new student
new_hours = [[2]]
prediction = model.predict(new_hours)

print("New Hours:", new_hours[0][0])
print("Predicted Result:", prediction[0])
```

Output:

```
New Hours: 2

Predicted Result: Fail
```

2) Regression

Regression is used when the output is a numeric value (continuous). The model predicts a number based on input features.
Examples of regression outputs include:

- Marks
- Salary
- House price
- Temperature
- Sales
- Profit

Regression is used whenever the final answer is a numeric value.

Example: Predict **Marks** from **Study Hours**. This is **supervised learning** because the dataset has **input (Hours)** and the **correct output (Marks)**.

```
from sklearn.linear_model import LinearRegression
import pandas as pd

# Training data (Hours -> Marks)
df = pd.DataFrame({
   "Hours": [1, 2, 3, 4, 5],
   "Marks": [30, 40, 50, 60, 70]
})

X = df[["Hours"]]   # input
y = df["Marks"]     # output (label)

# Train model
model = LinearRegression()
model.fit(X, y)

# Predict marks for a new student
new_hours = [[6]]
predicted_marks = model.predict(new_hours)

print("New Hours:", new_hours[0][0])
print("Predicted Marks:", predicted_marks[0])
```

Output:

```
New Hours: 6

Predicted Marks: 80.0
```

Note:

- If the output is a **category** (Pass/Fail) → supervised **classification**.
- If the output is a **number** (Marks) → supervised **regression**.

Real-life examples of supervised learning

- Predicting marks from study hours (Regression)
- Predicting house price from area, location, rooms (Regression)
- Detecting spam emails (Classification)

- Predicting whether a customer will buy a product or not (Classification)
- Detecting whether a transaction is fraud or genuine (Classification)

Widely used supervised learning algorithms are:

- Linear Regression (for regression)
- Logistic Regression (for classification)
- Decision Tree
- Random Forest
- K-Nearest Neighbors (KNN)
- Support Vector Machine (SVM)
- Naive Bayes
- Neural Networks / Deep Learning models

Supervised Learning Applications

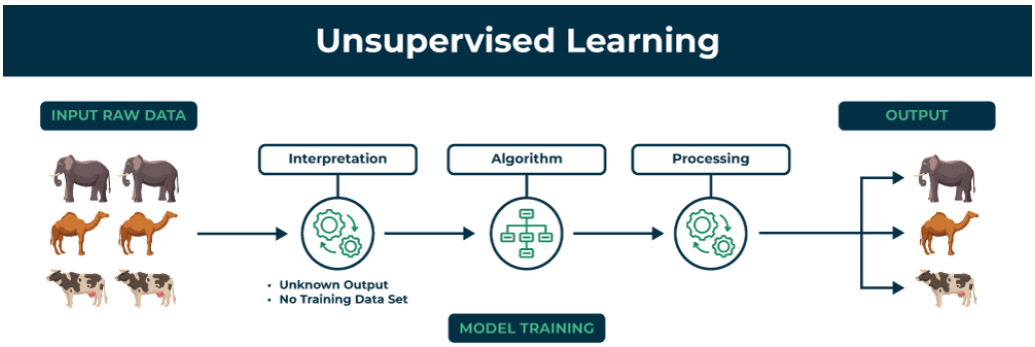
Supervised learning is used whenever the system needs to predict a known outcome (a category or a number).

- **Spam Filtering:**
  - **Task:** Classification (Spam or Not Spam).
  - **How it Works:** The model is trained on millions of emails manually labeled as spam or not spam. It learns to recognize patterns (keywords, sender behavior, links) that differentiate the two classes.
- **Predictive Maintenance:**
  - **Task:** Classification/Regression.
  - **How it Works:** Predicts when a piece of machinery (e.g., a jet engine component) is likely to fail by analyzing sensor data (temperature, vibration). This allows maintenance to be scheduled *before* a costly breakdown occurs.
- **Image Recognition:**
  - **Task:** Classification.
  - **How it Works:** Identifies what is in an image (e.g., recognizing faces, detecting objects, categorizing medical scans as cancerous or non-cancerous).

B) Unsupervised Learning

Unsupervised learning is a type of machine learning in which the dataset contains only input data (X) and there is no correct output column (Y). This means the data is unlabeled. Since there are no correct answers provided, the model cannot learn by comparing with labels. Instead, it discovers patterns and structure by analyzing similarities, differences, and relationships inside the dataset.

Unsupervised learning is commonly used when the main goal is not direct prediction, but to understand the dataset, discover groups, find hidden patterns, or reduce complexity. It is also widely used during EDA to explore data structure before applying supervised models. For example, unsupervised learning can analyze animal data and group the animals by their traits and behavior. These groups might represent different species which allows the machine to organize animals without any prior labels or categories.



Key characteristics of Unsupervised Learning

1. No labeled output is available. The dataset contains only input features.
2. The main goal is pattern discovery. The model finds groups, similarities, and relationships.
3. It is useful for exploration. It helps understand unknown data and supports EDA.
4. It is used when output is not available. Many datasets do not have a target column, so unsupervised methods are needed.

Main tasks in unsupervised learning

1) Clustering

Clustering means grouping similar items together based on similarity. The model automatically creates clusters, and items inside a cluster are more similar to each other than to items in other clusters. E.g. Grouping customers into segments based on purchasing behavior.

Example: Group students into 2 groups using only **Hours** and **Marks** (no labels like "Good/Weak" are given).

```
import pandas as pd
from sklearn.cluster import KMeans

df = pd.DataFrame({
   "Student": ["S1", "S2", "S3", "S4"],
   "Hours":   [1, 2, 7, 8],
   "Marks":   [35, 40, 75, 85]
})

print("=== Data (No labels given) ===")
print(df)

X = df[["Hours", "Marks"]]

model = KMeans(n_clusters=2, random_state=0,
n_init=10)
df["Cluster"] = model.fit_predict(X)

print("\n=== After Clustering ===")
print(df)
```

```
Output:
=== Data (No labels given) ===
  Student  Hours  Marks
0     S1     1     35
1     S2     2     40
2     S3     7     75
3     S4     8     85

=== After Clustering ===
  Student  Hours  Marks  Cluster
0     S1     1     35        1
1     S2     2     40        1
2     S3     7     75        0
3     S4     8     85        0

=== Cluster Centers ===
   Hours  Marks
0    7.5   80.0
1    1.5   37.5
```

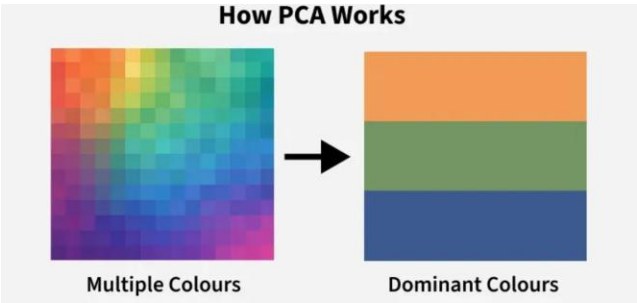
print("\n=== Cluster Centers ===") print(pd.DataFrame(model.cluster_centers_, columns=["Hours", "Marks"]))	

Interpretation:

- One cluster contains students with **low hours + low marks** (S1, S2).
- Another cluster contains students with **high hours + high marks** (S3, S4).

2) Dimensionality Reduction

Dimensionality reduction means reducing the number of input features while keeping the most useful information. This is helpful when a dataset has too many columns. It makes processing faster, improves visualization, and may reduce noise.
A common method is PCA (Principal Component Analysis).



Dataset: marks of students as follows:

Student	Math	Science	English
A	90	88	85
B	80	79	78
C	70	72	69
D	60	61	58

Here, all subjects move together:

High Math → High Science → High English. So, these 3 columns are giving almost the **same information**.

PCA will take following step: "Instead of keeping 3 similar columns, make **1 main column** that represents overall performance."

So PC1 becomes like an **Overall Score** (not exactly average, but similar idea).

- Students with high marks get high PC1
- Students with low marks get low PC1

import pandas as pd from sklearn.decomposition import PCA from sklearn.preprocessing import StandardScaler	**Output**: Original data (3 columns): Math Science English 0 90 88 85

```
df = pd.DataFrame({
   "Math":    [90, 80, 70, 60],
   "Science": [88, 79, 72, 61],
   "English": [85, 78, 69, 58]
})

print("Original data (3 columns):")
print(df)

# Step 1: Scale
X = StandardScaler().fit_transform(df)

# Step 2: Reduce 3 columns -> 1 column (PC1)
pca = PCA(n_components=1)
pc1 = pca.fit_transform(X)

df_pca = pd.DataFrame(pc1, columns=["PC1"])
print("\nAfter PCA (1 column):")
print(df_pca)

print("\nHow much information PC1 keeps:",
round(pca.explained_variance_ratio_[0]*100, 2),
"%")
```

```
1   80     79     78
2   70     72     69
3   60     61     58

After PCA (1 column):
       PC1
0 -2.248421
1 -0.806096
2  0.633430
3  2.421086

How much information PC1 keeps: 99.73 %
```

Interpretation:

- It will print **PC1 values** for each student.
- PC1 values are just a **new scoring system**.
- The line "PC1 keeps __% information" is the real key.
    - If it says **95%+**, it means 1 column is enough.

3) Association Rule Mining

Association rule mining finds items that occur together frequently. It is commonly used in market basket analysis. E.g.: Customers who buy bread often buy butter.

Association Rule Mining Formulas

Let the rule be: **A → B**
(where **A** and **B** are itemsets, e.g., {Tea} → {Sugar})

1) Support

$$\text{Support}(A \to B) = \text{Support}(A \cup B) = \frac{\#(A \cup B)}{N}$$

- $\#(A \cup B)$ = number of transactions containing **both A and B**
- N = total number of transactions

2) Confidence

$$\text{Confidence}(A \to B) = \frac{\text{Support}(A \cup B)}{\text{Support}(A)} = \frac{\#(A \cup B)}{\#(A)}$$

- $\#(A)$ = number of transactions containing **A**

3) Lift

$$\text{Lift}(A \to B) = \frac{\text{Confidence}(A \to B)}{\text{Support}(B)} = \frac{\text{Support}(A \cup B)}{\text{Support}(A) \times \text{Support}(B)}$$

- $\text{Support}(B) = \frac{\#(B)}{N}$

Interpretation of Lift:

- **Lift > 1**: positive association (B more likely when A occurs)
- **Lift = 1**: no association
- **Lift < 1**: negative association

Example for Association Rule Mining (Market Basket) For **Data (8 Shopping Baskets):**

Basket	Items Bought
T1	Milk, Bread
T2	Milk, Sugar, Tea
T3	Milk, Sugar, Coffee
T4	Tea, Sugar, Bread
T5	Milk, Sugar, Tea, Bread
T6	Coffee, Sugar
T7	Milk, Coffee, Bread
T8	Tea, Milk

Example Rule 1: Tea → Sugar

Meaning: "If Tea is bought, Sugar is often bought."

Step 1: Support

- Tea + Sugar appears in **T2, T4, T5** = **3 baskets**
- **Support = 3/8 = 0.375 = 37.5%**

Step 2: Confidence

- Tea appears in **T2, T4, T5, T8** = **4 baskets**
- **Confidence = 3/4 = 0.75 = 75%**

Step 3: Lift

- Sugar appears in **T2, T3, T4, T5, T6** = **5 baskets**

- Sugar support = **5/8 = 0.625**
- **Lift = 0.75 / 0.625 = 1.20**
  (Lift > 1 means Tea increases the chance of Sugar.)

Example Rule 2: Milk + Sugar → Tea

Meaning: "If Milk and Sugar are bought together, Tea is often bought."

- Milk+Sugar appears in **T2, T3, T5** = **3 baskets**
- Milk+Sugar+Tea appears in **T2, T5** = **2 baskets**

Support = 2/8 = 25%
Confidence = 2/3 = 66.7%
Tea support = Tea appears in 4 baskets ⇒ **4/8 = 0.5**
Lift = 0.667 / 0.5 = 1.33

Example

```
import pandas as pd

# ------------------------------
# 1) Simple transactions (Milk, Sugar, Tea, Coffee, Bread)
# ------------------------------
transactions = [
   {"Milk", "Bread"},              # T1
   {"Milk", "Sugar", "Tea"},       # T2
   {"Milk", "Sugar", "Coffee"},    # T3
   {"Tea", "Sugar", "Bread"},      # T4
   {"Milk", "Sugar", "Tea", "Bread"}, # T5
   {"Coffee", "Sugar"},            # T6
   {"Milk", "Coffee", "Bread"},    # T7
   {"Tea", "Milk"}                 # T8
]

N = len(transactions)

def count_contains(itemset):
   return sum(1 for t in transactions if itemset.issubset(t))

def support(itemset):
   return count_contains(itemset) / N

def confidence(A, B):
   return support(A | B) / support(A)

def lift(A, B):
   return confidence(A, B) / support(B)

# ------------------------------
# 2) Example rule: Tea -> Sugar
```

Output:

Rule	Support	Confidence	Lift
{'Tea'} -> {'Sugar'}	0.375	0.75	1.2

```
# ----------------------------
A = {"Tea"}
B = {"Sugar"}

sup = support(A | B)
conf = confidence(A, B)
lif = lift(A, B)

result = pd.DataFrame([{
   "Rule": f"{A} -> {B}",
   "Support": round(sup, 3),
   "Confidence": round(conf, 3),
   "Lift": round(lif, 3)
}])

print(result.to_string(index=False))
```

Real-life examples of unsupervised learning

- Customer segmentation for marketing (Clustering)
- Grouping students based on learning patterns (Clustering)
- Market basket analysis in supermarkets (Association rules)
- Reducing many survey features into fewer key features (Dimensionality reduction)

Widely used unsupervised learning algorithms are:

- K-Means Clustering
- Hierarchical Clustering
- DBSCAN
- PCA (Principal Component Analysis)
- Apriori Algorithm (for association rules)

Unsupervised Learning Applications

Unsupervised learning is essential for exploring new data, finding hidden insights, and organizing large amounts of information.

- **Customer Segmentation:**
  - **Task:** Clustering.
  - **How it Works:** Groups customers into distinct categories (e.g., high-spending, value-conscious, frequent shoppers) based on their purchasing habits and demographics without being explicitly told the categories beforehand. This enables highly targeted marketing campaigns.
- **Recommender Systems (The 'Hidden' Part):**
  - **Task:** Association/Clustering.
  - **How it Works:** While the final prediction (rating/recommendation) can be supervised, the initial step of grouping similar items or similar users (e.g., "People who watched X also watched Y") is often achieved through clustering or association rules on unlabeled user behavior data.
- **Anomaly Detection (e.g., Fraud):**

- **Task:** Clustering/Dimensionality Reduction.
- **How it Works:** The model learns what "normal" transactions or network behavior looks like. Any new activity that falls far outside the learned normal clusters is flagged as an anomaly or potential fraud.

Difference between Supervised & Unsupervised Learning

Supervised learning is used when the dataset contains correct output labels and the goal is to predict the output for new data. Unsupervised learning is used when labels are not available and the goal is to discover hidden patterns, groups, or structure from the dataset. Both approaches are important and are used depending on the nature of the data and the problem requirements.

Point	Supervised Learning	Unsupervised Learning
Meaning	Supervised learning is a type of machine learning where the dataset contains **inputs (X)** and also the **correct output (Y)**. The model learns by comparing its predictions with the correct answers and improving step by step.	Unsupervised learning is a type of machine learning where the dataset contains **only inputs (X)** and **no output (Y)**. The model learns by finding hidden patterns and structures without any given answers.
Data requirement	It requires **labeled data**, meaning each record has a target/output column.	It requires **unlabeled data**, meaning there is no target/output column.
Main objective	The main objective is to **predict the output** for new/unseen input data.	The main objective is to **discover patterns**, such as groups, similarities, or relationships in the data.
How learning happens	The model is trained using known examples and learns a mapping from input to output (**X → Y**). Errors are calculated and reduced during training.	The model analyzes the input data and automatically organizes it into meaningful structure, such as clusters or reduced features, based on similarity.
Common tasks	**Classification** (Spam/Not spam, Pass/Fail) and **Regression** (marks, salary, price prediction).	**Clustering** (grouping similar items), **Dimensionality reduction** (reducing features), and **Association rules** (finding items that occur together).
Examples	Predicting marks using study hours, predicting house price using area and location, detecting spam emails, fraud detection.	Grouping customers into segments, grouping students by performance style, market basket analysis, reducing many features using PCA.
Output	Produces a predicted label/class or numeric value.	Produces clusters/groups, reduced dimensions, or association patterns.
Conclusion	Supervised learning is used when correct output labels are available and prediction is required.	Unsupervised learning is used when labels are not available and pattern discovery is required.

2.4.2 Applications of ML in Real-World Scenarios

1. **Email Spam Detection (Classification)**
   Machine learning models learn from past emails labeled as spam or not spam. After training, they classify new incoming emails.
2. **Recommendation Systems**
   Online platforms recommend products, movies, music, or videos based on user history and behavior. ML learns what a user likes and suggests similar items.

3. **Fraud Detection in Banking**
   ML models detect unusual patterns in transactions, such as sudden high spending, unusual location, or repeated failed attempts. It helps identify possible fraud in real time.
4. **Face Recognition and Attendance Systems**
   ML models recognize faces from images and videos. This is used in security systems, phone unlocking, and attendance systems.
5. **Voice Assistants and Speech Recognition**
   ML helps convert speech into text and understand commands. It is used in voice assistants for tasks like calling, searching, and setting reminders.
6. **Medical Diagnosis Support**
   ML models can analyze medical reports, symptoms, and scan images to support doctors. It helps in risk prediction and early detection support.
7. **Customer Segmentation (Unsupervised learning)**
   Businesses group customers into different segments based on age, income, shopping style, and interests. This helps in targeted marketing.
8. **Demand Forecasting and Sales Prediction (Regression)**
   ML predicts future demand and sales using past sales data, season, trends, and market conditions. This helps in inventory planning and supply chain management.
9. **Social Media Analysis**
   ML is used to detect sentiment, trending topics, fake accounts, and abusive content. It is also used in emotion and sentiment analysis.
10. **Traffic Prediction and Route Optimization**
    ML helps estimate traffic levels and provides best route suggestions. It is used in navigation apps to save travel time.
11. **Image Classification and Object Detection**
    ML identifies objects in images such as cars, people, animals, and products. It is used in security cameras, self-driving cars, and industrial inspection.
12. **Predictive Maintenance in Industry**
    Machines generate sensor data. ML predicts when a machine may fail and helps plan maintenance early, reducing breakdown and repair cost.